Kotlin Lambdas

50 Examples from Basic to Advanced

A Comprehensive Guide to Lambda Expressions

Lambda Basics

Example 1: Simple Lambda Expression (Basic)

```
val greet = { println("Hello, World!") }
greet() // Output: Hello, World!
```

Explanation: A lambda with no parameters, enclosed in curly braces. Called like a function.

Q: How do you call a lambda that takes no parameters?

Example 2: Lambda with Single Parameter (Basic)

```
val square = { x: Int -> x * x }
println(square(5)) // Output: 25
```

Explanation: Lambda parameter before arrow (->), expression after. Type can be inferred.

Q: What separates parameters from the lambda body?

Example 3: Lambda with Multiple Parameters (Basic)

```
val add = { a: Int, b: Int -> a + b }
println(add(3, 7)) // Output: 10
```

Explanation: Multiple parameters separated by commas before the arrow.

Q: How do you define multiple parameters in a lambda?

Example 4: Lambda with Explicit Return Type (Basic)

```
val multiply: (Int, Int) -> Int = { x, y -> x * y }
println(multiply(4, 5)) // Output: 20
```

Explanation: Type annotation shows lambda takes two Ints and returns Int.

Q: What is the syntax for lambda type annotation?

Example 5: it: Implicit Single Parameter (Basic)

```
val double: (Int) -> Int = { it * 2 }
println(double(6)) // Output: 12
```

Explanation: Single parameter lambdas can use "it" instead of naming the parameter.

Q: When can you use "it" in a lambda?

Higher-Order Functions

Example 6: Function Taking Lambda (Basic)

```
fun operate(x: Int, y: Int, operation: (Int, Int) -> Int): Int {
```

```
return operation(x, y)
}
val result = operate(10, 5) { a, b -> a - b }
println(result) // Output: 5
```

Explanation: Functions can accept lambdas as parameters. Last lambda can be outside parentheses.

Q: What is a higher-order function?

Example 7: Function Returning Lambda (Basic)

```
fun makeMultiplier(factor: Int): (Int) -> Int {
return { number -> number * factor }
}
val timesThree = makeMultiplier(3)
println(timesThree(4)) // Output: 12
```

Explanation: Functions can return lambdas, creating closures over their parameters.

Q: What is a closure?

Collection Operations

Example 8: forEach with Lambda (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5)
numbers.forEach { println(it) }
// Output: 1 2 3 4 5 (each on new line)
```

Explanation: forEach iterates over collection, executing lambda for each element.

Q: How does forEach differ from a for loop?

Example 9: map Transformation (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5)
val doubled = numbers.map { it * 2 }
println(doubled) // Output: [2, 4, 6, 8, 10]
```

Explanation: map transforms each element using the lambda and returns new list.

Q: What does map return?

Example 10: filter Elements (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5, 6)
val evens = numbers.filter { it % 2 == 0 }
println(evens) // Output: [2, 4, 6]
```

Explanation: filter keeps only elements where lambda returns true.

Q: What type of value must a filter lambda return?

Example 11: any, all, none (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5)
println(numbers.any { it > 3 }) // true
println(numbers.all { it > 0 }) // true
println(numbers.none { it < 0 }) // true
```

Explanation: Predicates to check if any/all/none elements satisfy condition.

Q: What is the difference between any and all?

Example 12: find and firstOrNull (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5)
val firstEven = numbers.find { it % 2 == 0 }
println(firstEven) // Output: 2
val notFound = numbers.find { it > 10 }
println(notFound) // Output: null
```

Explanation: find returns first element matching predicate or null.

Q: What does find return if no element matches?

Example 13: reduce Accumulator (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5)
val sum = numbers.reduce { acc, num -> acc + num }
println(sum) // Output: 15
```

Explanation: reduce combines all elements using accumulator pattern.

Q: What is the difference between reduce and fold?

Example 14: sortedBy Custom Sorting (Basic)

```
data class Person(val name: String, val age: Int)
val people = listOf(
Person("Alice", 30),
Person("Bob", 25)
)
val sorted = people.sortedBy { it.age }
println(sorted) // Output: [Person(name=Bob, age=25), ...]
```

Explanation: sortedBy sorts collection based on lambda selector.

Q: How does sortedBy determine sort order?

Example 15: groupBy Classification (Basic)

```
val numbers = listOf(1, 2, 3, 4, 5, 6)
val grouped = numbers.groupBy { it % 2 == 0 }
println(grouped)
// Output: {false=[1, 3, 5], true=[2, 4, 6]}
```

Explanation: groupBy creates map grouping elements by lambda result.

Q: What type does groupBy return?

Lambda with Receivers

Example 16: Lambda with Receiver (apply) (Intermediate)

```
data class Person(var name: String = "", var age: Int = 0)
val person = Person().apply {
name = "Alice"
age = 30
}
println(person) // Output: Person(name=Alice, age=30)
```

Explanation: apply is a lambda with receiver (this). Returns the receiver object.

Q: What does apply return?

Example 17: Lambda with Receiver (also) (Intermediate)

```
val numbers = mutableListOf(1, 2, 3).also {
println("Initial list: $it")
it.add(4)
}
println(numbers) // Output: [1, 2, 3, 4]
```

Explanation: also passes receiver as lambda parameter (it), returns receiver.

Q: How does also differ from apply?

Example 18: Lambda with Receiver (let) (Intermediate)

```
val name: String? = "Alice"
val length = name?.let {
println("Processing: $it")
it.length
}
println(length) // Output: 5
```

Explanation: let is useful for null-safe calls and transformations.

Q: What does let return?

Example 19: Lambda with Receiver (run) (Intermediate)

```
val result = "Hello".run {
println("Length: $length")
uppercase()
}
println(result) // Output: HELLO
```

Explanation: run executes lambda on receiver and returns lambda result.

Q: How does run differ from let?

Example 20: Lambda with Receiver (with) (Intermediate)

```
data class Config(var host: String = "", var port: Int = 0)
val config = Config()
with(config) {
host = "localhost"
```

```
    port = 8080
    }
    println(config) // Output: Config(host=localhost, port=8080)
```

Explanation: with is not an extension, takes receiver as first parameter.

Q: When would you use with instead of apply?

Advanced Collection Operations

Example 21: flatMap Transformation (Intermediate)

```
val lists = listOf(listOf(1, 2), listOf(3, 4), listOf(5))
val flattened = lists.flatMap { it.map { num -> num * 2 } }
println(flattened) // Output: [2, 4, 6, 8, 10]
```

Explanation: flatMap maps and flattens in one operation.

Q: How does flatMap differ from map?

Example 22: fold with Initial Value (Intermediate)

```
val numbers = listOf(1, 2, 3, 4, 5)
val product = numbers.fold(1) { acc, num -> acc * num }
println(product) // Output: 120
val concat = listOf("a", "b", "c").fold("") { acc, s -> acc + s }
println(concat) // Output: abc
```

Explanation: fold takes initial value, more flexible than reduce.

Q: Why might you prefer fold over reduce?

Example 23: partition Split (Intermediate)

```
val numbers = listOf(1, 2, 3, 4, 5, 6)
val (evens, odds) = numbers.partition { it % 2 == 0 }
println("Evens: $evens") // [2, 4, 6]
println("Odds: $odds") // [1, 3, 5]
```

Explanation: partition splits collection into two based on predicate.

Q: What type does partition return?

Example 24: associate Key-Value Pairs (Intermediate)

```
val numbers = listOf(1, 2, 3, 4)
val squared = numbers.associate { it to it * it }
println(squared) // Output: {1=1, 2=4, 3=9, 4=16}
val nameMap = numbers.associateWith { "Number $it" }
println(nameMap) // {1=Number 1, 2=Number 2, ...}
```

Explanation: associate creates map from lambda-generated pairs.

Q: What is the difference between associate and associateWith?

Example 25: takeWhile and dropWhile (Intermediate)

```
val numbers = listOf(1, 2, 3, 4, 5, 2, 1)
val taken = numbers.takeWhile { it < 4 }
println(taken) // Output: [1, 2, 3]
val dropped = numbers.dropWhile { it < 4 }
println(dropped) // Output: [4, 5, 2, 1]
```

Explanation: takeWhile takes elements until predicate fails, dropWhile drops until it fails.

Q: When does takeWhile stop taking elements?

Example 26: windowed Sliding Window (Intermediate)

```
val numbers = listOf(1, 2, 3, 4, 5)
val windows = numbers.windowed(3) { it.sum() }
println(windows) // Output: [6, 9, 12]
val pairs = numbers.zipWithNext { a, b -> a + b }
println(pairs) // Output: [3, 5, 7, 9]
```

Explanation: windowed creates sliding windows, can transform each window.

Q: What does the size parameter in windowed represent?

Closures

Example 27: Closure Capturing Variables (Intermediate)

```
fun makeCounter(): () -> Int {
var count = 0
return { ++count }
}
val counter = makeCounter()
println(counter()) // 1
println(counter()) // 2
println(counter()) // 3
```

Explanation: Lambda captures and maintains access to variables from enclosing scope.

Q: What is the lifecycle of captured variables?

Example 28: Closure Modifying External State (Intermediate)

```
var sum = 0
val numbers = listOf(1, 2, 3, 4, 5)
numbers.forEach { sum += it }
println(sum) // Output: 15
```

Explanation: Lambdas can modify variables from outer scope (mutable closures).

Q: What are the risks of mutable closures?

Type Aliases

Example 29: Type Alias for Lambda (Intermediate)

```
typealias Predicate<T> = (T) -> Boolean
typealias Transformer<T, R> = (T) -> R
fun <T> List<T>.customFilter(predicate: Predicate<T>): List<T> {
return this.filter(predicate)
}
val numbers = listOf(1, 2, 3, 4, 5)
val evens = numbers.customFilter { it % 2 == 0 }
println(evens) // [2, 4]
```

Explanation: Type aliases make complex lambda types more readable.

Q: When should you use type aliases for lambdas?

Extension Lambdas

Example 30: Extension Function with Lambda (Intermediate)

```
fun <T> List<T>.customMap(transform: (T) -> T): List<T> {
val result = mutableListOf<T>()
for (item in this) {
result.add(transform(item))
}
return result
}
val numbers = listOf(1, 2, 3)
val doubled = numbers.customMap { it * 2 }
println(doubled) // [2, 4, 6]
```

Explanation: Extension functions can take lambdas as parameters.

Q: How do extension functions enhance lambda usage?

Lambda Returning Lambda

Example 31: Currying with Lambdas (Intermediate)

```
val add: (Int) -> (Int) -> Int = { a -> { b -> a + b } }
val addFive = add(5)
println(addFive(3)) // Output: 8
println(addFive(10)) // Output: 15
// Or directly
println(add(2)(3)) // Output: 5
```

Explanation: Lambdas can return other lambdas, enabling currying.

Q: What is currying and what are its benefits?

Inline Lambdas

Example 32: Inline Functions (Intermediate)

```
inline fun <T> measure(block: () -> T): Pair<T, Long> {
val start = System.currentTimeMillis()
```

```
val result = block()
val time = System.currentTimeMillis() - start
return result to time
}
val (result, time) = measure {
(1..1000000).sum()
}
println("Result: $result, Time: ${time}ms")
```

Explanation: inline eliminates lambda overhead by inlining code at call site.

Q: What is the performance benefit of inline functions?

Example 33: noinline Modifier (Intermediate)

```
inline fun doSomething(
action: () -> Unit,
noinline logger: () -> Unit
) {
val loggerRef = logger // Can store noinline lambda
action()
logger()
}
doSomething(
action = { println("Action") },
logger = { println("Log") }
)
```

Explanation: noinline prevents specific lambda from being inlined.

Q: When would you use noinline?

Example 34: crossinline Modifier (Intermediate)

```
inline fun runAsync(crossinline action: () -> Unit) {
Thread {
action() // Can call in different context
}.start()
}
runAsync {
println("Running in thread: ${Thread.currentThread().name}")
}
```

Explanation: crossinline allows lambda to be called in non-local context.

Q: What is the difference between crossinline and noinline?

Sequence Operations

Example 35: Lazy Sequences (Intermediate)

```
val numbers = listOf(1, 2, 3, 4, 5)
// Eager evaluation
val eager = numbers.map { it * 2 }.filter { it > 5 }
// Lazy evaluation
val lazy = numbers.asSequence()
.map {
println("Mapping $it")
it * 2
}
.filter {
```

```
println("Filtering $it")
it > 5
}
.toList()
println(lazy) // Output: [6, 8, 10]
```

Explanation: Sequences evaluate lambdas lazily, processing one element at a time.

Q: When should you use sequences instead of collections?

DSL Building

Example 36: HTML DSL with Lambdas (Advanced)

```
class HTML {
private val content = StringBuilder()
fun body(init: BODY.() -> Unit) {
val body = BODY()
body.init()
content.append(body.toString())
}
override fun toString() = "<html>$content</html>"
}
class BODY {
private val content = StringBuilder()
fun h1(text: String) {
content.append("<h1>$text</h1>")
}
override fun toString() = "<body>$content</body>"
}
fun html(init: HTML.() -> Unit): HTML {
val html = HTML()
html.init()
return html
}
val page = html {
body {
h1("Hello, DSL!")
}
}
println(page)
```

Explanation: Lambda with receiver enables type-safe DSL construction.

Q: How do lambda receivers enable DSL syntax?

Example 37: Builder Pattern with Lambdas (Advanced)

```
class HttpRequest {
var url: String = ""
var method: String = "GET"
val headers = mutableMapOf<String, String>()
var body: String = ""
fun header(key: String, value: String) {
headers[key] = value
}
}
fun request(init: HttpRequest.() -> Unit): HttpRequest {
return HttpRequest().apply(init)
}
val req = request {
url = "https://api.example.com"
method = "POST"
header("Content-Type", "application/json")
body = """{"key": "value"}"""
```

```
}
```

Explanation: Builder pattern using lambda with receiver for clean configuration.

Q: What are the advantages of builder pattern with lambdas?

Function Composition

Example 38: Function Composition (Advanced)

```
infix fun <A, B, C> ((B) -> C).compose(f: (A) -> B): (A) -> C {
return { a -> this(f(a)) }
}
val addOne: (Int) -> Int = { it + 1 }
val double: (Int) -> Int = { it * 2 }
val square: (Int) -> Int = { it * it }
val composed = square compose double compose addOne
println(composed(3)) // (3+1)*2^2 = 64
// Or using andThen
infix fun <A, B, C> ((A) -> B).andThen(f: (B) -> C): (A) -> C {
return { a -> f(this(a)) }
}
val pipeline = addOne andThen double andThen square
println(pipeline(3)) // ((3+1)*2)^2 = 64
```

Explanation: Composing functions creates new functions from existing ones.

Q: What is the difference between compose and andThen?

Memoization

Example 39: Memoization with Lambdas (Advanced)

```
fun <T, R> memoize(fn: (T) -> R): (T) -> R {
val cache = mutableMapOf<T, R>()
return { arg ->
cache.getOrPut(arg) { fn(arg) }
}
}
val fibonacci: (Int) -> Long = memoize { n ->
when (n) {
0 -> 0L
1 -> 1L
else -> fibonacci(n - 1) + fibonacci(n - 2)
}
}
println(fibonacci(40)) // Much faster with memoization!
```

Explanation: Memoization caches function results to avoid repeated computation.

Q: What are the trade-offs of memoization?

Recursion

Example 40: Recursive Lambda (Advanced)

```
val factorial: (Int) -> Long = { n ->
if (n <= 1) 1L else n * factorial(n - 1)
}
println(factorial(5)) // 120
// Y-combinator for anonymous recursion
fun <T, R> fix(f: ((T) -> R) -> (T) -> R): (T) -> R {
return { t -> f(fix(f))(t) }
}
val fact = fix<Int, Long> { recurse ->
{ n -> if (n <= 1) 1L else n * recurse(n - 1) }
}
println(fact(5)) // 120
```

Explanation: Lambdas can be recursive, Y-combinator enables anonymous recursion.

Q: Why do we need special techniques for anonymous recursion?

Functional Patterns

Example 41: Monadic Operations (Advanced)

```
sealed class Result<out T> {
data class Success<T>(val value: T) : Result<T>()
data class Failure(val error: String) : Result<Nothing>()
fun <R> map(f: (T) -> R): Result<R> = when (this) {
is Success -> Success(f(value))
is Failure -> this
}
fun <R> flatMap(f: (T) -> Result<R>): Result<R> = when (this) {
is Success -> f(value)
is Failure -> this
}
}
fun divide(a: Int, b: Int): Result<Int> =
if (b == 0) Result.Failure("Division by zero")
else Result.Success(a / b)
val result = divide(10, 2)
.flatMap { divide(it, 0) }
.map { it * 2 }
println(result) // Failure(Division by zero)
```

Explanation: Monadic operations chain computations that may fail.

Q: What is the difference between map and flatMap?

Example 42: Either Type with Lambdas (Advanced)

```
sealed class Either<out L, out R> {
data class Left<L>(val value: L) : Either<L, Nothing>()
data class Right<R>(val value: R) : Either<Nothing, R>()
fun <T> fold(left: (L) -> T, right: (R) -> T): T = when (this) {
is Left -> left(value)
is Right -> right(value)
}
}
fun parseAge(input: String): Either<String, Int> =
input.toIntOrNull()
?.let { if (it >= 0) Either.Right(it) else Either.Left("Negative age") }
?: Either.Left("Invalid number")
val age = parseAge("25")
val message = age.fold(
left = { error -> "Error: $error" },
right = { value -> "Age is $value" }
)
println(message) // Age is 25
```

Explanation: Either type represents success or failure with values of different types.

Q: How does Either differ from Result or Optional?

Lazy Evaluation

Example 43: Lazy Initialization (Advanced)

```
class ExpensiveObject {
init {
println("Creating expensive object...")
Thread.sleep(1000)
}
fun use() = println("Using object")
}
val lazyValue: ExpensiveObject by lazy {
println("Initializing...")
ExpensiveObject()
}
println("Before first access")
lazyValue.use() // Initialization happens here
lazyValue.use() // No re-initialization
// Custom lazy
fun <T> lazyOf(initializer: () -> T): Lazy<T> =
object : Lazy<T> {
private var cached: T? = null
override val value: T
get() {
if (cached == null) cached = initializer()
return cached!!
}
override fun isInitialized() = cached != null
}
```

Explanation: lazy delegates initialization to first access using lambda.

Q: What are the thread-safety modes of lazy?

Coroutines

Example 44: Suspending Lambdas (Advanced)

```
import kotlinx.coroutines.*
suspend fun fetchData(): String {
delay(1000)
return "Data"
}
fun processAsync(block: suspend () -> Unit) {
GlobalScope.launch {
block()
}
}
// Usage
processAsync {
val data = fetchData()
println(data)
}
// Custom suspend builder
suspend fun <T> retry(
times: Int,
block: suspend () -> T
```

```
): T {
repeat(times - 1) {
try {
return block()
} catch (e: Exception) {
// Retry
}
}
return block() // Last attempt
}
```

Explanation: Suspending lambdas enable coroutine-based async operations.

Q: What makes a lambda suspending?

Context Receivers

Example 45: Context Receivers (Kotlin 1.6.20+) (Advanced)

```
// Experimental feature
interface Logger {
fun log(message: String)
}
interface Database {
fun query(sql: String): List<String>
}
context(Logger, Database)
fun performOperation() {
log("Starting operation")
val results = query("SELECT * FROM users")
log("Found ${results.size} users")
}
// Usage with context
class ConsoleLogger : Logger {
override fun log(message: String) = println("[LOG] $message")
}
class MockDatabase : Database {
override fun query(sql: String) = listOf("User1", "User2")
}
with(ConsoleLogger()) {
with(MockDatabase()) {
performOperation()
}
}
```

Explanation: Context receivers allow functions to require multiple contexts.

Q: How do context receivers differ from regular receivers?

Delegated Properties

Example 46: Custom Delegate with Lambdas (Advanced)

```
import kotlin.reflect.KProperty
class Validated<T>(
private val initialValue: T,
private val validator: (T) -> Boolean
) {
private var value = initialValue
operator fun getValue(thisRef: Any?, property: KProperty<*>): T {
return value
}
```

```
operator fun setValue(thisRef: Any?, property: KProperty<*>, newValue: T) {
if (validator(newValue)) {
value = newValue
} else {
throw IllegalArgumentException("Invalid value: $newValue")
}
}
}
class Person {
var age: Int by Validated(0) { it >= 0 && it <= 150 }
var name: String by Validated("") { it.isNotBlank() }
}
val person = Person()
person.age = 25
// person.age = -5 // Throws exception
```

Explanation: Custom property delegates can use lambdas for validation logic.

Q: What are the operator functions required for delegation?

Type-Safe Builders

Example 47: Advanced DSL with @DslMarker (Advanced)

```
@DslMarker
annotation class TableDsl
@TableDsl
class Table(val name: String) {
val columns = mutableListOf<Column>()
fun column(name: String, type: String, init: Column.() -> Unit = {}) {
columns.add(Column(name, type).apply(init))
}
}
@TableDsl
class Column(val name: String, val type: String) {
var nullable: Boolean = true
var default: String? = null
fun index() {
println("Creating index on $name")
}
}
fun table(name: String, init: Table.() -> Unit): Table {
return Table(name).apply(init)
}
val usersTable = table("users") {
column("id", "INTEGER") {
nullable = false
}
column("name", "VARCHAR") {
default = "'Unknown'"
index()
}
}
```

Explanation: @DslMarker prevents implicit receiver nesting in DSLs.

Q: What problem does @DslMarker solve?

Variance and Lambdas

Example 48: Contravariant Lambdas (Advanced)

```
// Contravariant in parameter type
interface Comparator<in T> {
fun compare(a: T, b: T): Int
}
open class Animal(val name: String)
class Dog(name: String, val breed: String) : Animal(name)
val animalComparator: Comparator<Animal> = object : Comparator<Animal> {
override fun compare(a: Animal, b: Animal): Int {
return a.name.compareTo(b.name)
}
}
// Can use Animal comparator for Dogs (contravariance)
val dogComparator: Comparator<Dog> = animalComparator
// With lambdas
fun <T> sortWith(items: List<T>, comparator: (T, T) -> Int): List<T> {
return items.sortedWith { a, b -> comparator(a, b) }
}
val dogs = listOf(Dog("Rex", "Terrier"), Dog("Max", "Labrador"))
val sorted = sortWith(dogs) { a, b -> a.name.compareTo(b.name) }
```

Explanation: Lambda parameter types are contravariant (accept supertypes).

Q: Why are function parameter types contravariant?

Reflection

Example 49: Lambda Reflection (Advanced)

```
import kotlin.reflect.KFunction
import kotlin.reflect.full.*
fun greet(name: String, age: Int): String {
return "Hello $name, age $age"
}
val functionRef: KFunction2<String, Int, String> = ::greet
println(functionRef.name) // greet
println(functionRef.parameters.map { it.name }) // [name, age]
println(functionRef.returnType) // String
val result = functionRef.call("Alice", 30)
println(result) // Hello Alice, age 30
// Lambda type checking
val lambda: (Int) -> Int = { it * 2 }
val lambdaRef = lambda::class
// Higher-order function reflection
fun measure(block: () -> Unit): Long {
val start = System.currentTimeMillis()
block()
return System.currentTimeMillis() - start
}
val measureRef = ::measure
println(measureRef.parameters.first().type) // () -> Unit
```

Explanation: Kotlin reflection allows runtime inspection of lambda types.

Q: What are the limitations of lambda reflection?

Advanced Patterns

Example 50: Continuation-Passing Style (CPS) (Advanced)

```
// Traditional recursive fibonacci
fun fib(n: Int): Int {
return if (n <= 1) n else fib(n - 1) + fib(n - 2)
```

```
}
// CPS version - tail recursive
fun fibCPS(n: Int, cont: (Int) -> Int): Int {
return if (n <= 1) {
cont(n)
} else {
fibCPS(n - 1) { result1 ->
fibCPS(n - 2) { result2 ->
cont(result1 + result2)
}
}
}
}
println(fibCPS(10) { it }) // 55
// Async-like CPS
fun <T, R> asyncCPS(
value: T,
computation: (T) -> R,
callback: (R) -> Unit
) {
Thread {
val result = computation(value)
callback(result)
}.start()
}
asyncCPS(5, { it * it }) { result ->
println("Result: $result") // 25
}
// State machine simulation
sealed class State
object Initial : State()
data class Running(val value: Int) : State()
object Done : State()
fun stateMachine(
state: State,
next: (State) -> Unit
) {
when (state) {
Initial -> next(Running(0))
is Running -> {
if (state.value < 5) {
next(Running(state.value + 1))
} else {
next(Done)
}
}
Done -> println("Finished")
}
}
```

Explanation: CPS transforms control flow into explicit continuation functions.

Q: What are the advantages and use cases of CPS?

Lambda Practice Questions

1. What is a lambda expression in Kotlin and how does it differ from an anonymous function?
2. Explain the concept of 'it' in Kotlin lambdas. When can and can't you use it?
3. What is a higher-order function? Provide examples of built-in higher-order functions.
4. Describe the difference between map, flatMap, and forEach.
5. What is a closure? How do lambdas capture variables from their enclosing scope?
6. Explain lambda with receiver. Compare apply, also, let, run, and with.
7. What is the difference between fold and reduce? When would you use each?
8. How do inline functions improve lambda performance? What are noinline and crossinline?
9. Explain lazy sequences and when they provide better performance than eager evaluation.
10. What is function composition? How can you implement compose and andThen operators?
11. How do you create type-safe builders (DSLs) using lambdas with receivers?
12. What is the purpose of @DslMarker annotation?
13. Explain memoization and implement a memoization function for lambdas.
14. How do you handle recursion in lambdas? What is the Y-combinator?
15. What are suspending lambdas and how are they used with coroutines?
16. Explain contravariance in lambda parameter types.
17. How can you use lambdas to implement the builder pattern?
18. What is continuation-passing style (CPS) and when would you use it?
19. How do you create custom property delegates using lambdas?
20. What are the differences between function references (::) and lambda expressions?
21. How does Kotlin handle SAM (Single Abstract Method) conversions with lambdas?
22. Explain the partition function and provide a use case.
23. What is the difference between groupBy and associateBy?
24. How can you chain multiple lambdas for data transformation pipelines?

25. What are the performance implications of using lambdas vs regular functions?

26. How do you test functions that accept lambdas as parameters?

27. Explain how to use lambdas for event handling and callbacks.

28. What is the difference between a lambda and a function type?

29. How can lambdas be used to implement functional patterns like Either or Result?

30. What are best practices for naming lambda parameters in complex scenarios?